Practical - 13

Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Theory:

1. Do PHP endpoints return valid JSON responses?

   Answer: Yes, PHP endpoints return valid JSON responses using json_encode(). The frontend receives and processes the JSON data.

2. Are GET and POST requests handled properly?

   Answer: Yes, GET requests are used to display student records, while POST requests are used to add, update, and delete student records.

3. Are loading, success, and error states shown on the frontend?

   Answer: Yes, the frontend displays loading messages, success messages after operations, and error messages when something goes wrong.

4. Is input validation performed before database operations?

   Answer: Yes, the program checks that all required fields are filled and that the email address is valid before performing database operations.

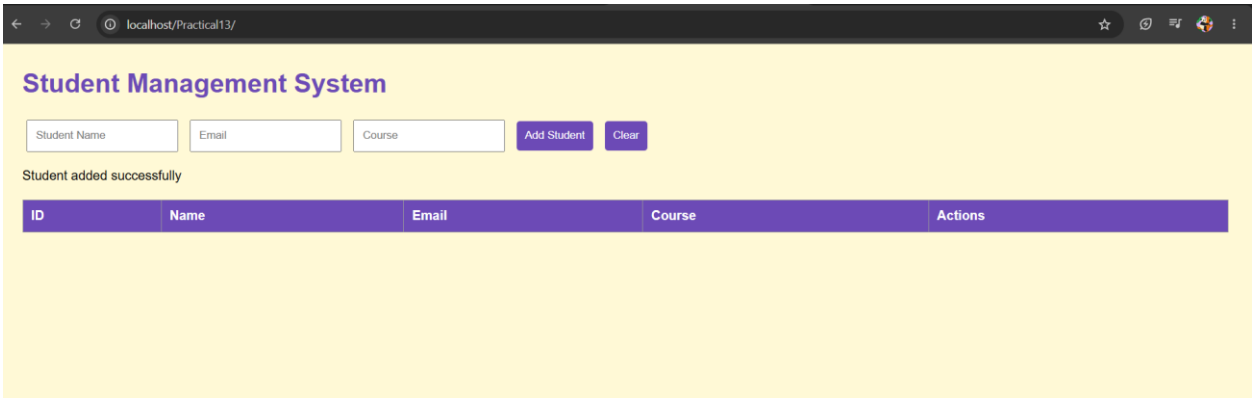
A. Implementation/ Output/Result **(Keep only one option which is suitable to practical)**

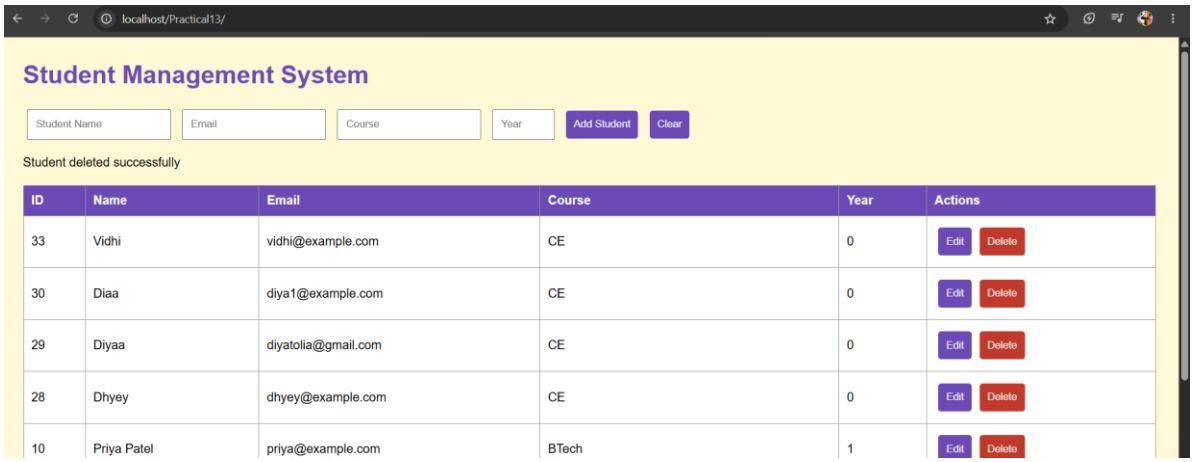
Tags Used and Their Purpose

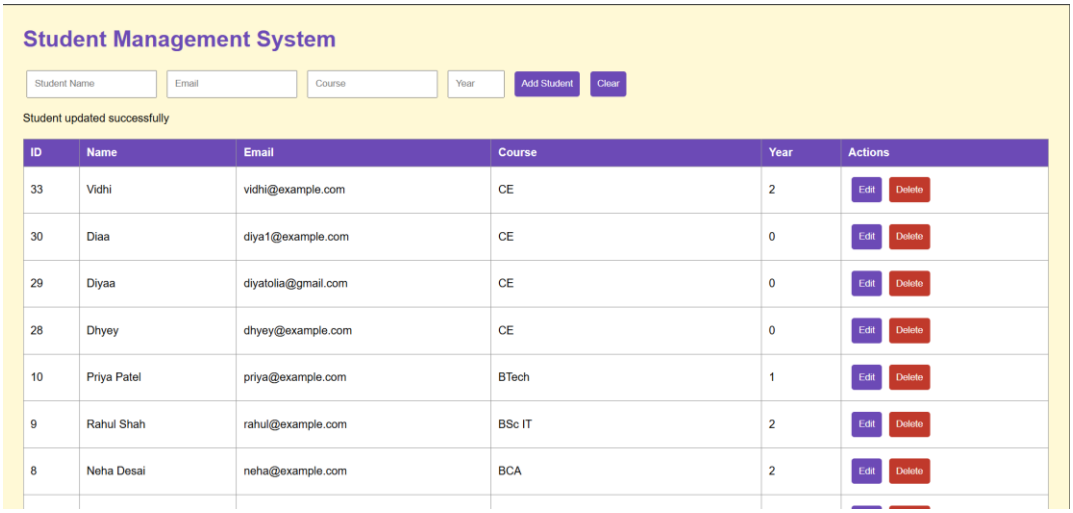
Tag	Purpose
`<html>`	Defines the HTML document.
`<head>`	Contains page information and styles.
`<title>`	Sets the webpage title.
`<style>`	Adds CSS styling to the webpage.
`<body>`	Contains the visible webpage content.
`<h1>`	Displays the main heading.

`<form>`	Creates the student input form.
`<input>`	Accepts student details.
`<button>`	Performs add, update, clear, and delete actions.
`<p>`	Displays loading, success, and error messages.
`<table>`	Displays student records.
`<thead>`	Defines the table headings.
`<tbody>`	Contains student records added dynamically.
`<tr>`	Defines a table row.
`<th>`	Defines a table heading cell.
`<td>`	Defines a table data cell.
`<script>`	Contains JavaScript code for Fetch API and CRUD operations.

B. Evidence of Work (Screenshots)







C. GitHub Repository Link:

https://github.com/diyatolia1212/StudentHub/tree/59681b7c3354e13e448dab02feff3b0fbe32b924/practical13

D. Challenges Faced:

- Connecting PHP API endpoints with the MySQL database.
- Sending and receiving data using JSON and Fetch API.
- Performing CRUD operations without reloading the webpage.
- Handling validation errors and displaying appropriate messages.

Learning Outcomes:

- Learned how to create PHP JSON API endpoints.
- Learned how to use Fetch API for asynchronous communication.
- Understood how to perform add, view, update, and delete operations without page reload.
- Learned how to validate input and handle success and error responses.

Faculty Remarks: (Keep this blank for faculty)